THE ODIN PROJECT

INTERMEDIATE HTML & CSS

Full Stack JavaScript Learning Path

Presented by Tejus Sanly

Subject: Computer Systems | Group: 4230z

Course roadmap

01 Intermediate HTML

SVG graphics and tables

02 Intermediate CSS

Units, selectors, styling, and positioning

03 Forms

Inputs, labels, and validation

04 CSS Grid

Two-dimensional responsive layouts

05 Projects

Sign-up Form and Admin Dashboard

Why learn intermediate HTML & CSS?

- Move beyond basic static web pages.
- Create semantic, accessible forms and structured data.
- Build adaptable layouts with CSS Grid and Flexbox.
- Use maintainable styles and responsive design techniques.

HTML concepts: SVG and tables

SVG (Scalable Vector Graphics)

Resolution-independent vector graphics. Useful for logos, icons, and illustrations. Can be styled using CSS.

HTML tables

Use <table>, <tr>, <th>, and <td> for genuinely tabular data. Add captions and clear headings for accessibility.

Choose HTML elements based on meaning, not just appearance.

CSS foundations: units and typography

Relative units

rem and em scale with font sizes; percentages depend on their context.

Viewport units

vw and vh relate to viewport dimensions. Useful with care in responsive layouts.

Text styling

Control fonts, line heights, spacing, shadows, and visual hierarchy.

Design goal: readable content on mobile and desktop screens.

Powerful selectors and positioning

```css
/* Select a button on hover */
button:hover { background: #0ea5a4; }

/* Position an element within a card */
.card { position: relative; }
.badge { position: absolute; top: 1rem; right: 1rem; }
```

Key ideas

Pseudo-classes and combinators target elements. Positioning includes static, relative, absolute, fixed, and sticky.

CSS functions and custom properties

```
:root{
 --accent:#14b8a6;
 --space:1rem ;
}
.container{
 width:min(90% ,1100px);
 padding:calc(var(--space)* 2);
 color:var(--accent);
}
```

Why they matter

CSS variables keep values consistent. Functions such as calc(), min(), and max() create flexible sizing rules.

Browser compatibility and tools

- Default browser styles affect element appearance.
- Check compatibility before using newer CSS features.
- Use browser developer tools to inspect layouts and debug styles.
- Frameworks and preprocessors can speed up large projects.

HTML forms: collecting user input

```
<form>
 <label for="email">Email</label>
 <input id="email" type="email" required>
 <label for="password">Password</label>
 <input id="password" type="password"
    minlength="8" required>
 <button type="submit">Register</button>
</form>
```

Good form design

Connect labels with inputs. Choose correct input types. Give useful feedback and keep forms easy to navigate.

Form validation

Built-in HTML validation

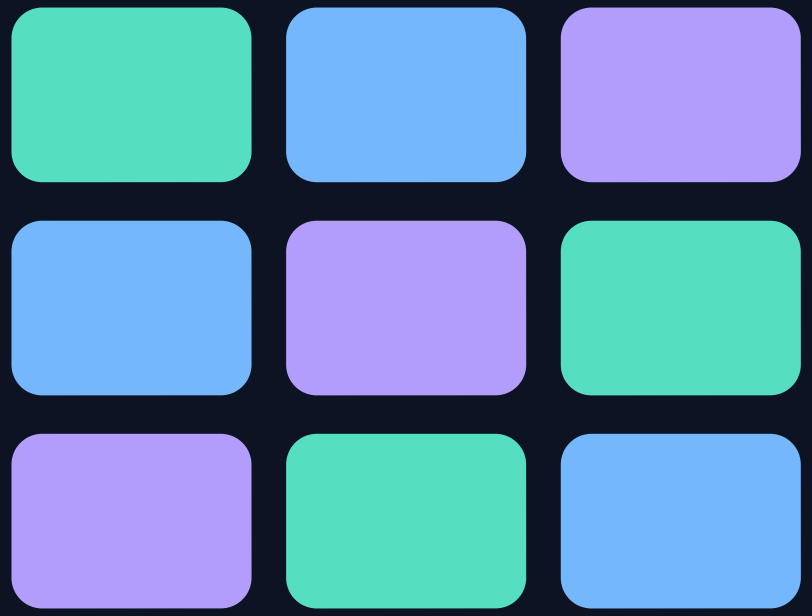
required, minlength, maxlength, pattern, type, min, and max help check user input.

User experience

Make errors understandable. Explain the expected format. Remember: server-side validation is still essential.

Example: type="email" checks basic email formatting in the browser.

CSS Grid: two-dimensional layouts



```css
.dashboard {
 display: grid;
 grid-template-columns: repeat(3, 1fr);
 gap: 1rem;
}
```

Grid organizes rows and columns. Use fr units, repeat(), gaps, and named areas to build layouts.

Responsive layouts: Grid + Flexbox

CSS Grid

Best for two-dimensional page sections such as dashboards, cards, and complex content placement.

Flexbox

Best for one-dimensional alignment such as navigation menus, toolbars, and groups of buttons.

Often the best interface uses both layout methods together.

Practice projects from the course

Project 1 — Sign-up Form

Build a registration interface with clear labels, input styling, helpful constraints, and validation feedback.

Project 2 — Admin Dashboard

Build a multi-section dashboard using Grid for the main layout and Flexbox for component alignment.

Key takeaways

- HTML provides structure and meaning.
- CSS provides styling, layout, and responsive behavior.
- Forms make pages interactive and accessible.
- CSS Grid and Flexbox work together for modern interfaces.
- Real projects help turn theory into practical skills.

References & thank you

Thank you!

Course source:

The Odin Project — Intermediate HTML and CSS

https://www.theodinproject.com/paths/full-stack-javascript/courses/intermediate-html-and-css

Presented by Tejus Sanly | Computer Systems | Group 4230z